

Void: Designer-First XR UI Platform

Void: Designer-First XR UI Platform

Complete Product Overview & Problem Analysis

Project Title: Void

Core Vision

Void is a designer-first platform for creating spatial user interfaces and complete XR experiences without requiring game engine expertise. It democratizes XR app creation by providing an intuitive, code-free environment for designing, prototyping, and deploying XR applications.

Problems We're Solving

1. XR App Creation Barrier

Current State: To create XR apps, designers must learn Unity or Unreal Engine

- Requires C# or Blueprint scripting knowledge
- Complex development environments not built for designers
- Steep learning curve prevents UI/UX designers from entering XR space

Impact: Talented designers can't participate in XR development, limiting innovation and design quality

2. UI Creation Complexity (Unity-Specific Pain Points)

Problems Experienced:

- **Button creation requires coding:** Simple buttons need naming conventions, scripts, manual hover state programming
- **Styling is code-heavy:** Basic visual changes require CSS-like coding or inspector work
- **Learning curve is steep:** Constant YouTube tutorial watching for basic UI tasks
- **No visual feedback:** Hard to preview what UI actually looks like in XR context

Impact: Hours wasted on basic UI tasks instead of focusing on experience design

3. Screen Linking & Prototyping Nightmare

Current Problems:

- **No visual prototyping:** Can't wire "Button A → Screen B" without coding
- **Script-heavy interactions:** Every connection requires custom C# scripts
- **Back-and-forth workflow:** Constant switching between UI editor and code editor
- **No flow preview:** Can't test user journeys without building entire project

Impact: Prototyping takes forever, design iterations are painful

4. Event Integration Complexity

Specific Issues:

- **UI to scene events:** Connecting button to "bounce ball" requires script knowledge
- **Limited event system:** Hard to trigger animations, object interactions, or scene changes
- **No visual event wiring:** Everything must be coded manually
- **Platform limitations:** Each XR platform has different event systems

Impact: Simple interactions become development bottlenecks

5. Interaction Method Switching Hell

Problems:

- **Controller vs hand tracking:** Completely different implementation approaches
- **Poke interactions:** Separate setup process for finger-based UI interactions
- **Gaze-based UI:** Another entirely different system to learn and implement
- **Platform differences:** Meta Quest vs Vision Pro have different interaction paradigms

Impact: Designers can't easily test different interaction methods, limiting UX exploration

6. Multi-Platform Deployment Complexity

Current State:

- **Platform-specific builds:** Unity build process differs for each XR device
- **Performance optimization:** Each platform requires different optimization strategies
- **Testing bottlenecks:** Must build and deploy to test on actual devices
- **Distribution complexity:** App stores, sideloading, enterprise deployment all different

Impact: Massive development overhead for simple experiences

7. Spatial Testing & Context Issues

Current Problems:

- **No spatial context during design:** Designing UI on flat screens without understanding how it feels in 3D space
- **Testing requires full builds:** Can't quickly test UI positioning, scale, and spatial relationships

- **Environment integration:** Hard to test how UI works in different spatial contexts (rooms, outdoor spaces, etc.)

Impact: UI that looks good on screen feels wrong in actual XR environments

Target Users & Use Cases

Primary Users

1. **UI/UX Designers** transitioning to XR who want to create spatial interfaces without learning game engines
2. **Product Designers** exploring XR applications for their companies
3. **Creative Technologists** who understand design but don't want to become Unity experts
4. **Unity Beginners** who want to focus on world-building instead of UI coding

Use Cases

- **Meditation/wellness apps** for Vision Pro
 - **Productivity tools** for Meta Quest
 - **Art gallery experiences** for any XR platform
 - **Educational apps** with spatial UI components
 - **Game UI systems** that integrate with Unity experiences
-

Void Solution

Two-Track Platform Approach

1. **Standalone XR Apps** → Design in Void, deploy directly to headsets (no Unity required)
2. **Game Engine Integration** → Design in Void, export to Unity/Unreal for complex experiences

Core Features

Visual UI Builder

- Figma-like interface for spatial UI design with real-time 3D preview
- Component library for XR UI patterns
- Easy styling without code
- Spatial positioning tools (distance, height, angle relative to user)

Interaction Prototyping

- Drag-and-drop screen linking and visual event wiring
- Multi-input support: Controller, hand tracking, gaze, and poke interactions
- Real-time interaction testing without coding

Spatial Testing & Preview System

Two Testing Approaches:

Option 1: Virtual Environment Testing

- Built-in 3D environments (rooms, outdoor spaces, etc.)
- Place UI in virtual space during design
- Preview shows UI in same virtual environment on headset
- Provides spatial context without needing real environment setup

Option 2: Real Environment Anchoring

- Connects to headset's room setup/guardian system
- Anchor UI elements to real-world positions (walls, desks, floating in space)
- UI appears exactly where positioned in actual room during testing

Live Preview Workflow:

1. Design UI screens in Void on computer
2. Set spatial positioning (distance, height, angle from user)
3. Hit "Preview on Quest/Vision Pro"
4. Void streams experience wirelessly to headset
5. Interact with UI in real 3D space
6. Make adjustments in Void, see changes instantly on headset

Multi-Platform Export

- Deploy directly to Meta Quest, Vision Pro, etc.
- Export to Unity/Unreal as prefabs or packages
- Platform-specific optimization handled automatically

Technical Architecture

- **Design Layer:** Browser-based or desktop app with 3D spatial canvas
 - **Preview Engine:** Real-time XR streaming and testing system
 - **Spatial Engine:** Environment integration and anchoring system
 - **Export Engine:** Multiple output targets (standalone apps, Unity packages, etc.)
 - **Sync System:** Live wireless connection between design tool and XR devices
-

Development Phases

Phase 1 (MVP - Class Project)

- Visual UI builder with basic spatial layout tools
- Simple screen linking and prototyping
- Meta Quest 3 wireless preview and testing
- Basic virtual environment testing
- Unity integration for advanced use cases

Phase 2 (Platform Expansion)

- Vision Pro support with real environment anchoring
- Standalone app deployment (no Unity required)
- Advanced interaction systems and gestures
- Unreal Engine integration
- Multi-environment testing (indoor/outdoor/custom)

Phase 3 (Ecosystem)

- Multi-platform deployment pipeline
- Real-time collaboration features
- Asset marketplace and component library
- Enterprise tools and team workflows
- Advanced spatial anchoring and persistence

Success Criteria

1. **Usability:** Figma-experienced designer can create first XR app prototype in under 2 hours
2. **Technical:** Void-created UIs perform equivalently to hand-coded Unity UI
3. **Spatial Quality:** Designers can accurately preview and test spatial UI positioning before deployment
4. **Adoption:** XR design community adopts Void as standard tool for spatial UI design
5. **Business:** Platform supports both hobbyist creators and enterprise XR development teams

Competitive Landscape

Current Tools Analysis:

- **Unity UI Builder:** Visual but still requires scripting for interactions, no live XR preview
- **Figma to Unity plugins:** Static asset export only, no interaction logic or spatial testing
- **Playmaker:** Visual scripting but not UI-specific, no spatial design tools
- **Coherent UI:** HTML/CSS based but developer-focused, limited XR integration

Gap Identified: No tool combines visual spatial UI design + interaction prototyping + live XR testing + multi-platform deployment in a designer-first package.

Simple Explanation

Void - Complete Simple Explanation

What you're building: A web-based design tool where UI/UX designers can create interfaces for XR apps AND easily connect them to game events without coding.

The core problem you experienced: When making UI in Unity for your XR project:

- Creating buttons required coding
- Linking buttons to other screens needed scripts
- **Connecting UI to game events was hell** - like making a "Start Game" button actually start your experience, or a button that makes a ball bounce
- Switching interaction methods (controller/hand) was complex

What Void does differently:

1. **Visual UI creation** - Drag and drop like Figma, but for 3D space
2. **Easy screen linking** - "Button A goes to Screen B" with no coding
3. **Visual event linking** - Connect buttons directly to Unity events: "Start Experience," "Bounce Ball," "Play Animation," etc.
4. **Interaction switching** - Toggle controller/hand/gaze with checkboxes
5. **Live testing** - Preview on actual Meta Quest while designing
6. **Unity bridge** - Export everything (screens + event connections) to Unity

Key example workflows:

- Design a menu with "Start," "Settings," "Quit" buttons
- "Start" button → triggers your Unity game/experience to begin
- "Settings" button → goes to settings screen
- "Quit" button → ends the application
- All connected visually, no C# scripts needed